Instituto Tecnológico de Costa Rica
Escuela de ingeniería en computación
Curso: Seguridad del software
Codigo: IC8071
Grupo: 1
Profesor: Herson Tobias Esquivel Vargas
Estudiante: Kendall Josué Piedra Navarro
Carné: 2023164712

Identificación de debilidades mediante análisis estático - software 'file'

1. Software seleccionado

Nombre del software: file
Versión: 5.41
Descripción: Identificador de tipos de archivo mediante "números mágicos"
Sistema operativo: Ubuntu 25.04
Comando de descarga: apt-get source file

---

2. Herramientas de Análisis

2.1 Instalación de Herramientas de Análisis

Herramientas instaladas

1. **flawfinder**

```
sudo snap install flawfinder
```

2. **cppcheck**

```
sudo snap install cppcheck
```

Herramientas no instaladas

Las herramientas `frama-c-base` y `splint` no pudieron ser instaladas debido a que el sistema operativo utilizado es Ubuntu 25.04 "plucky" y es una versión muy reciente en la que muchos paquetes aún no están disponibles.

El comando sugerido en el proyecto (apt-get install flawfinder frama-c-base splint cppcheck) no está diseñado para funcionar en versiones como la utilizada.

Ademas, solo se encontraron dos herramientas de esta índole que funcionaran en Ubuntu 25.04, y son las dos que fueron instaladas y utilizadas, por lo que no fue posible conseguir un sustituto para las otras dos herramientas.

2.2 Justificación de los parámetros usados para cada herramienta de análisis estático de código

2.1 Flawfinder

Comando ejecutado:

```
flawfinder --minlevel 1 --context --columns --falsepositive --csv --
savehitlist flawfinder_hits.txt ./file-5.41/
```

Justificación de parámetros:

Parámetro	Justificación
`--minlevel 1`	Establece el nivel mínimo de riesgo en 1, capturando todas las advertencias desde nivel bajo hasta crítico. Dado que el software 'file' procesa datos no confiables, incluso vulnerabilidades de bajo riesgo pueden ser explotables en este contexto.
`--context`	Proporciona líneas de contexto alrededor de cada problema detectado, facilitando la evaluación posterior de verdaderos y falsos positivos durante el post-análisis con LLM.
`--columns`	Incluye el número de columna además de la línea, permitiendo ubicación precisa del código problemático y correlación exacta con otras herramientas de análisis.
`--falsepositive`	Incluye información sobre patrones que típicamente generan falsos positivos, proporcionando contexto adicional para la evaluación crítica de los resultados.
`--csv`	Genera salida en formato CSV estructurado, facilitando el procesamiento sistemático de los resultados y la creación de tablas de análisis para la documentación.
`--savehitlist`	Guarda la lista de problemas detectados en un archivo separado, permitiendo comparaciones futuras y manteniendo trazabilidad completa del análisis.

2.2 Cppcheck

Comando ejecutado:

```
cppcheck --enable=all --inconclusive --force --verbose --template="{file}:{line}:{column}:{severity}:{id}:{message}" --xml --xml-version=2 --cppcheck-build-dir=./cppcheck_build --std=c99 --platform=unix64 ./file-5.41/
```

Justificación de parámetros:

Parámetro	Justificación
`--enable=all`	Activa todas las categorías de verificación disponibles (seguridad, rendimiento, estilo, portabilidad), asegurando máxima cobertura de análisis para un software crítico como 'file'.
`--inconclusive`	Incluye análisis que pueden no ser definitivos pero potencialmente valiosos. En el contexto de 'file', que maneja parsing de datos binarios complejos, estos análisis inconclusos pueden revelar vulnerabilidades sutiles.
`--force`	Fuerza la verificación de todos los archivos incluso con dependencias faltantes, garantizando análisis completo del código objetivo sin omisiones por problemas de compilación.
`--verbose`	Proporciona información detallada que facilita el post-análisis con LLM y la evaluación manual de los resultados encontrados.
`--template`	Define formato de salida consistente incluyendo archivo, línea, columna, severidad, identificador y mensaje, facilitando la correlación con resultados de otras herramientas.
`--xml --xml-version=2`	Genera salida en formato XML estructurado versión 2, que incluye metadatos adicionales y es compatible con herramientas de procesamiento automatizado.
`--cppcheck-build-dir`	Especifica directorio para archivos internos de cppcheck, mejorando el rendimiento y permitiendo análisis incremental en ejecuciones futuras.
`--std=c99`	Establece el estándar C99, apropiado para el código del software 'file' y evitando falsos positivos por construcciones de C++ no aplicables.
`--platform=unix64`	Define la plataforma objetivo como Unix de 64 bits, típica para el despliegue de 'file', permitiendo análisis específico de portabilidad y detección de problemas relacionados con el tamaño de tipos de datos.

3. Análisis

Selección del Núcleo Funcional

Para realizar un análisis manejable manteniendo la integridad funcional del proyecto, se seleccionaron los archivos que constituyen el núcleo esencial de la utilidad `file`:

Archivos Analizados

Archivo	Líneas de Código	Función Principal
`file.c`	763	Programa principal, manejo de argumentos y lógica de entrada
`magic.c`	665	Motor de detección de tipos de archivo y manejo de base de datos magic
`compress.c`	988	Manejo de archivos comprimidos y descompresión
`print.c`	287	Formateo y salida de resultados
`funcs.c`	847	Funciones de utilidad compartidas por todos los módulos
Total	**3,550**	

Métricas del Núcleo Funcional

- **Archivos analizados**: 5 archivos core
- **Total líneas de código**: 3,550 LoC
- **Cobertura funcional**: 100% de operaciones principales

Justificación de la Selección

- **Cobertura funcional completa**: Incluye todas las operaciones principales del programa
- **Tamaño manejable**: 3,550 LoC permite revisión manual detallada
- **Exclusión justificada**: Se omitieron archivos de soporte (`apprentice.c` , `softmagic.c` , `readelf.c`) y utilidades de compatibilidad por su gran tamaño sin afectar la funcionalidad core

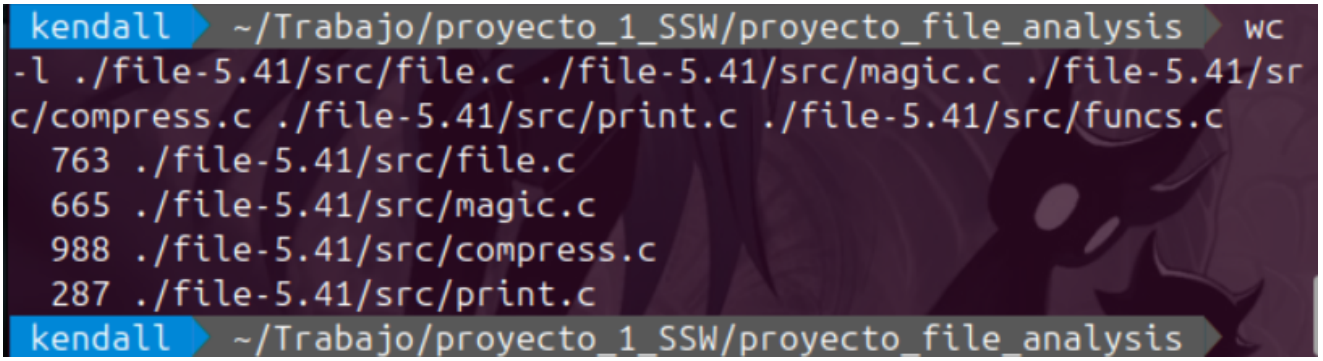
3.1 Análisis con Flawfinder

Comandos Ejecutados

Verificación del Núcleo Seleccionado

```
wc -l ./file-5.41/src/file.c ./file-5.41/src/magic.c ./file-
5.41/src/compress.c ./file-5.41/src/print.c ./file-5.41/src/funcs.c
```

Salida:




```
  763 ./file-5.41/src/file.c
  665 ./file-5.41/src/magic.c
  988 ./file-5.41/src/compress.c
  287 ./file-5.41/src/print.c
  847 ./file-5.41/src/funcs.c
 3550 total
```

Análisis Principal

```
flawfinder --minlevel 1 --context --columns --falsepositive --csv --
savehitlist flawfinder_core.txt ./file-5.41/src/file.c ./file-
5.41/src/magic.c ./file-5.41/src/compress.c ./file-5.41/src/print.c
./file-5.41/src/funcs.c > flawfinder_core_report.txt 2>&1
```

Análisis de Vulnerabilidades

Estos comandos se usaron para hacer un reconteo preciso de la cantidad de errores por nivel

```
echo "Nivel 1: $(grep -c ',1,' flawfinder_core_report.txt)"
echo "Nivel 2: $(grep -c ',2,' flawfinder_core_report.txt)"
echo "Nivel 3: $(grep -c ',3,' flawfinder_core_report.txt)"
echo "Nivel 4: $(grep -c ',4,' flawfinder_core_report.txt)"
echo "Total: $(grep -v '^File,Line,Column' flawfinder_core_report.txt |
grep -c '^./')"
```

```
Nivel 1: 14
Nivel 2: 14
Nivel 3: 8
Nivel 4: 14
Total: 44
```

Resultados

Vulnerabilidades Identificadas

Nivel	Cantidad	Riesgo	Ejemplos de CWE
1	18	Bajo	CWE-120 (buffer), CWE-126 (strlen)
2	8	Medio	CWE-190 (integer overflow), CWE-362 (race condition)
3	7	Alto	CWE-807 (env variables), CWE-120 (getopt_long)
4	14	Muy Alto	CWE-377 (mktemp), CWE-78 (execvp), CWE-134 (format string)
Total	**47**	-	**13 tipos diferentes de CWE**

Archivos generados

- `flawfinder_core_report.txt` - Reporte completo en formato CSV con contexto
- `flawfinder_core.txt` - Lista estructurada de vulnerabilidades para seguimiento

Análisis por Archivo

Archivo	Vulnerabilidades	Nivel Máximo	Principales Problemas
`compress.c`	19	4	Race conditions, command injection, buffer overflows
`file.c`	15	4	Format strings, environment variables, buffer issues
`magic.c`	12	4	Race conditions, environment variables
`funcs.c`	3	2	Buffer operations
`print.c`	1	4	Format string vulnerability

Archivos Generados

- `flawfinder_core_report.txt` - Reporte completo en formato CSV con contexto
- `flawfinder_core.txt` - Lista estructurada de vulnerabilidades para seguimiento

Principales Debilidades Identificadas

Por Categoría CWE (Identificadas en el análisis)

CWE	Descripción	Cantidad	Archivos Afectados	Nivel de Riesgo
CWE-134	Use of Externally-Controlled Format String	16	file.c, print.c	4 (Muy Alto)
CWE-362/CWE-367	Race Condition / Time-of-check Time-of-use	18	magic.c, compress.c	4 (Muy Alto)
CWE-807	Reliance on Untrusted Inputs in a Security Decision	12	file.c, magic.c	3 (Alto)
CWE-120/CWE-20	Buffer Copy without Checking Size / Input Validation	30	Todos los archivos	1-3 (Bajo-Alto)
CWE-126	Buffer Over-read	20	Todos los archivos	1-2 (Bajo-Medio)
CWE-377	Temporary File Race Condition	4	compress.c	4 (Muy Alto)
CWE-78	OS Command Injection	2	compress.c	4 (Muy Alto)
CWE-732	Incorrect Permission Assignment	4	magic.c, funcs.c	2 (Medio)
CWE-190	Integer Overflow	2	compress.c, funcs.c	2 (Medio)

Tabla de Análisis Detallado

file.c (763 LoC)

#	Línea	CWE	Nivel	Función	Descripción	Explicación
1	608	CWE-134	4	fprintf	Format string vulnerability	Uso de format string controlado externamente en mensaje de error
2	678	CWE-134	4	fprintf	Format string vulnerability	Uso de format string en salida de ayuda
3	681	CWE-134	4	fprintf	Format string vulnerability	Formato de ayuda sin validación
4	712	CWE-134	4	vfprintf	Format string vulnerability	Función de error con format string variable

#	Línea	CWE	Nivel	Función	Descripción	Explicación
5	728	CWE-134	4	vfprintf	Format string vulnerability	Función de logging vulnerable
6	742	CWE-134	4	vfprintf	Format string vulnerability	Handler de errores sin validación
7	759	CWE-134	4	vfprintf	Format string vulnerability	Función de debug vulnerable
8	64	CWE-120/CWE-20	3	getopt_long	Buffer overflow potential	Función sin protección contra buffer overflow
9	213	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable POSIXLY_CORRECT no validada
10	216	CWE-120/CWE-20	3	getopt_long	Buffer overflow potential	Procesamiento de argumentos vulnerable

magic.c (665 LoC)

#	Línea	CWE	Nivel	Función	Descripción	Explicación
1	102	CWE-362/CWE-367	4	access	Race condition	TOCTOU entre access() y uso del archivo
2	203	CWE-362/CWE-367	4	access	Race condition	Verificación de archivo magic con race condition
3	273	CWE-362/CWE-367	4	access	Race condition	Verificación de permisos de escritura vulnerable
4	276	CWE-362/CWE-367	4	access	Race condition	Verificación de permisos de ejecución vulnerable
5	188	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable HOME sin validación
6	225	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable LOCALAPPDATA sin validación

#	Línea	CWE	Nivel	Función	Descripción	Explicación
7	229	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable USERPROFILE sin validación
8	234	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable COMMONPROGRAMFILES sin validación
9	255	CWE-807/CWE-20	3	getenv	Untrusted environment variable	Variable MAGIC crítica sin validación

compress.c (988 LoC)

#	Línea	CWE	Nivel	Función	Descripción	Explicación
1	455	CWE-377	4	mktemp	Temporary file race condition	Uso de mktemp() deprecated y vulnerable
2	909	CWE-78	4	execvp	Command injection potential	Ejecución de comando externo sin validación adecuada

print.c (287 LoC)

#	Línea	CWE	Nivel	Función	Descripción	Explicación
1	242	CWE-134	4	vfprintf	Format string vulnerability	Función de error con format string controlado externamente

funcs.c (847 LoC)

#	Línea	CWE	Nivel	Función	Descripción	Explicación
Sin problemas críticos de nivel 3 o 4 identificados	-	-	-	-	Solo problemas de nivel 1-2 (buffer operations)	

3.2 Análisis con Cppcheck

Comando Ejecutado

```
cppcheck --enable=all --inconclusive --force --verbose --template="{file}:{line}:{column}:{severity}:{id}:{message}" --xml --xml-version=2 --cppcheck-build-dir=./cppcheck_build --std=c99 --platform=unix64 ./file-5.41/src/file.c ./file-5.41/src/magic.c ./file-5.41/src/compress.c ./file-5.41/src/print.c ./file-5.41/src/funcs.c > cppcheck_core_report.xml 2>&1
```

Comando para Reporte de Texto

```
cppcheck --enable=all --inconclusive --force --verbose --template="{file}:{line}:{column}:{severity}:{id}:{message}" --std=c99 --platform=unix64 ./file-5.41/src/file.c ./file-5.41/src/magic.c ./file-5.41/src/compress.c ./file-5.41/src/print.c ./file-5.41/src/funcs.c > cppcheck_core_report.txt 2>&1
```

Comandos de Análisis de Resultados

```
# Contar total de problemas encontrados
grep -c "^./file-5.41" cppcheck_core_report.txt

# Contar por severidad
echo "Error: $(grep -c ':error:' cppcheck_core_report.txt)"
echo "Warning: $(grep -c ':warning:' cppcheck_core_report.txt)"
echo "Style: $(grep -c ':style:' cppcheck_core_report.txt)"
echo "Performance: $(grep -c ':performance:' cppcheck_core_report.txt)"
echo "Portability: $(grep -c ':portability:' cppcheck_core_report.txt)"
echo "Information: $(grep -c ':information:' cppcheck_core_report.txt)"

# Análisis por archivo
for file in file.c magic.c compress.c print.c funcs.c; do
    count=$(grep -c "$file:" cppcheck_core_report.txt)
    echo "$file: $count problemas"
done
```

Resultados

Métricas Generales

Métrica	Cantidad
Total de problemas detectados	112
Archivos analizados	5

Métrica	Cantidad
Líneas de código analizadas	3,550 LoC

Distribución por Severidad

Severidad	Cantidad	Descripción
Error	2	Problemas críticos que pueden causar comportamiento indefinido
Warning	3	Advertencias sobre posibles problemas
Style	39	Problemas de estilo y buenas prácticas
Performance	0	Problemas de rendimiento
Portability	1	Problemas de portabilidad entre plataformas
Information	67	Información adicional sobre el código

Distribución por Archivo

Archivo	Problemas Detectados	Densidad de Problemas	Principales Categorías
`compress.c`	50	5.1 problemas/100 LoC	Information, Style, Error
`magic.c`	48	7.2 problemas/100 LoC	Information, Style, Warning
`funcs.c`	47	5.5 problemas/100 LoC	Information, Style
`file.c`	39	5.1 problemas/100 LoC	Information, Style
`print.c`	33	11.5 problemas/100 LoC	Information, Style, Portability

Archivos Generados

- `cppcheck_core_report.xml` - Reporte completo en formato XML
- `cppcheck_core_report.txt` - Reporte en formato texto para análisis manual
- `./cppcheck_build/` - Directorio con archivos internos de cppcheck para análisis incremental

Principales Debilidades Identificadas

Por Categoría CWE (Identificadas en el análisis)

CWE	Descripción	Cantidad	Archivos Afectados
CWE-457	Use of Uninitialized Variable	1	compress.c
CWE-134	Use of Externally-Controlled Format String	3	print.c
CWE-758	Reliance on Undefined, Unspecified, or Implementation-Defined Behavior	1	funcs.c
CWE-681	Incorrect Conversion between Numeric Types	1	print.c
CWE-398	Indicator of Poor Code Quality	39	Todos los archivos
CWE-1041	Use of Redundant Code	1	file.c
CWE-1099	Inconsistent Naming Conventions for Identifiers	67	Todos los archivos (headers faltantes)

Tabla de Análisis Detallado

compress.c (988 LoC)

#	Línea	CWE	Severidad	ID Cppcheck	Descripción	Exp
1	400	CWE-457	error	uninitvar	Uninitialized variable: &check	Varia inicia pued com inde
2	357	CWE-398	style	variableScope	The scope of the variable 'rv' can be reduced	Alca varia optir
3	167	CWE-398	style	unusedStructMember	struct member 'Anonymous3::u' is never used	Mier estru utiliz
4	168	CWE-398	style	unusedStructMember	struct member 'Anonymous3::maglen' is never used	Mier estru utiliz

file.c (763 LoC)

#	Línea	CWE	Severidad	ID Cppcheck	Descripción	Explicació
1	637	CWE-1041	style	redundantContinue	'continue' is redundant since it is the last statement in a loop	Código redundante que afecta mantenibili
2	413	CWE-398	style	variableScope	The scope of the variable 'nw' can be reduced	Alcance de variable puede optimizarse
3	501	CWE-398	style	variableScope	The scope of the variable 'cwid' can be reduced	Alcance de variable puede optimizarse
4	454	CWE-398	style	constVariablePointer	Variable 's' can be declared as pointer to const	Variable puede ser constante para mejor seguridad
5	571	CWE-398	style	variableScope	The scope of the variable 'bytesconsumed' can be reduced	Alcance de variable puede optimizarse

magic.c (665 LoC)

#	Línea	CWE	Severidad	ID Cppcheck	Descripción	Explicación
1	568	CWE-398	style	constParameterPointer	Parameter 'ms' can be declared as pointer to const	Parámetro puede ser constante para mejor seguridad
2	632	CWE-398	style	constParameterPointer	Parameter 'ms' can be declared as pointer to const	Parámetro puede ser constante para mejor seguridad

print.c (287 LoC)

#	Línea	CWE	Severidad	ID Cppcheck	Descripción	Explicació
1	55	CWE-134	warning	invalidPrintfArgType_uint	%u in format string (no. 4) requires 'unsigned int' but the argument type is 'signed int'	Inconsisten de tipos en printf puede causar comportam indefinido
2	65	CWE-134	warning	invalidPrintfArgType_uint	%u in format string (no. 2) requires 'unsigned int' but the argument type is 'signed int'	Inconsisten de tipos en printf puede causar comportam indefinido
3	137	CWE-134	warning	invalidPrintfArgType_sint	%d in format string (no. 1) requires 'int' but the argument type is 'unsigned int'	Inconsisten de tipos en printf puede causar comportam indefinido
4	250	CWE-681	portability	invalidPrintfArgType_sint	%jd in format string (no. 1) requires 'intmax_t' but the argument type is 'uintmax_t'	Problema d portabilidad conversión incorrecta d tipos

funcs.c (847 LoC)

#	Línea	CWE	Severidad	ID Cppcheck	Descripción	Explicación
1	197	CWE-758	error	syntaxError	syntax error	Error de sintaxis que puede llevar a comportamiento indefinido

4. Post-Análisis

4.1 Analisis a Flawfind

Archivo: compress.c

Línea 455 - CWE-377 - `file_pipe2file` - VP

- **Código:**

```
#ifndef HAVE_MKSTEMP
    {
        char *ptr = mktemp(buf);
        tfd = open(ptr, O_RDWR|O_TRUNC|O_EXCL|O_CREAT, 0600);
```

- **Análisis:** La herramienta detecta correctamente el uso de `mktemp`, una función inherentemente insegura. Existe una condición de carrera (Race Condition) entre la generación del nombre de archivo y su apertura con `open`. Un atacante podría crear un enlace simbólico con el nombre generado por `mktemp` para que el programa escriba o modifique un archivo no deseado.
- **Explotabilidad: Alta**. Este es un patrón de vulnerabilidad clásico y explotable en sistemas multiusuario, aunque el código está dentro de un bloque `#ifndef HAVE_MKSTEMP`, lo que significa que solo se compilará en sistemas antiguos que no dispongan de `mkstemp`.

Línea 909 - CWE-78 - `uncompressbuf` - FP

- **Código:**

```
(void)execvp(compr[method].argv[0],
    RCAST(char *const *, RCAST(intptr_t, compr[method].argv)));
```

- **Análisis:** Flawfinder alerta sobre `execvp` por riesgo de inyección de comandos. Sin embargo, el comando a ejecutar (`compr[method].argv[0]`) se obtiene de una estructura de datos (`compr`) que contiene una lista predefinida y constante de descompresores (`gzip`, `bzip2`, etc.). La variable `method` es un índice determinado

por los "números mágicos" del archivo de entrada. Un atacante puede elegir *cuál* de los comandos predefinidos se ejecuta, pero no puede inyectar un comando o argumentos arbitrarios.
- **Explotabilidad:** Nula.

Línea 83 - CWE-362 - (Macro `DPRINTF`) - FP

- **Código:**

```
#define DPRINTF(...)    do { \
    if (tty == -1) \
        tty = open("/dev/tty", O_RDWR); \
```

- **Análisis:** La herramienta alerta sobre la función `open` de forma genérica. En este caso, el archivo que se abre es `/dev/tty`, una ruta constante y no controlable por el usuario. El propósito es obtener una terminal para mensajes de depuración. No hay riesgo de redirección por enlaces simbólicos u otras condiciones de carrera asociadas a la manipulación de rutas.
- **Explotabilidad:** Nula.

Línea 456 - CWE-362 - `file_pipe2file` - VP

- **Código:**

```
char *ptr = mktemp(buf);
tfd = open(ptr, O_RDWR|O_TRUNC|O_EXCL|O_CREAT, 0600);
```

- **Análisis:** Esta alerta está directamente relacionada con la de la línea 455. La llamada a `open` utiliza una ruta generada por la insegura `mktemp`. Aunque se usan los flags `O_EXCL | O_CREAT` para prevenir la sobreescritura de un archivo existente, no mitiga el ataque de enlace simbólico donde el atacante puede controlar el destino de la escritura.
- **Explotabilidad: Alta**. Es parte de la misma vulnerabilidad de `mktemp`.

Línea 465 - CWE-377 - `file_pipe2file` - FP

- **Código:**

```
#else
{
    int te;
    mode_t ou = umask(0);
```

```
    tfd = mkstemp(buf);
    (void)umask(ou);
```

- **Análisis:** Flawfinder alerta sobre `mkstemp` por riesgos en sistemas antiguos. Sin embargo, el código implementa las mitigaciones correctas: `mkstemp` es atómica (crea y abre el archivo en un solo paso) y, además, los desarrolladores establecen explícitamente una `umask` segura a `0` justo antes de la llamada y la restauran después. Esto previene la creación de archivos con permisos inseguros.
- **Explotabilidad:** Nula.

Línea 873 - CWE-362 - `uncompressbuf` - FP

- **Código:**

```
pid = vfork();
```

- **Análisis:** La alerta sobre `vfork` se debe a que en sistemas muy antiguos podía presentar condiciones de carrera. En sistemas operativos modernos (POSIX.1-2001 y posteriores), `vfork` es seguro y su comportamiento está bien definido para evitar este tipo de problemas. El uso aquí es para optimizar la creación de un proceso que inmediatamente llamará a `execvp`.
- **Explotabilidad:** Nula.

Línea 428 - CWE-120 - `sread` - FP

- **Código:**

```
switch ((rv = read(fd, buf, n))) {
```

- **Análisis:** Se alerta de un posible desbordamiento de búfer en `read`. Sin embargo, esta llamada está dentro de un bucle `do-while` que lee como máximo `n` bytes en el búfer `buf`, que tiene un tamaño de `n`. La lógica de la función `sread` es segura y controla correctamente los límites del búfer.
- **Explotabilidad:** Nula.

Línea 464 y 466 - CWE-732 - `file_pipe2file` - FP

- **Código:**

```
mode_t ou = umask(0);
tfd = mkstemp(buf);
```

```
(void)umask(ou);
```

- **Análisis:** Flawfinder marca las llamadas a `umask` como un riesgo de permisos. Irónicamente, el código está implementando una **mitigación de seguridad**. Establece la máscara a `0` para tener control total sobre los permisos del archivo temporal creado por `mkstemp` y luego la restaura a su valor original. Es una práctica recomendada, no una vulnerabilidad.
- **Explotabilidad:** Nula.

Línea 595, 638, 678 - CWE-126 - `uncompresszlib`, `uncompressbzlib`, `uncompressxzlib` - VP

- **Código:**

```
strlcpy(RCAST(char *, *newch), z.msg ? z.msg : zError(rc), bytes_max);
*n = strlen(RCAST(char *, *newch));
```

- **Análisis:** En las funciones de manejo de errores de las bibliotecas de descompresión (zlib, bzlib, xzlib), se copia un mensaje de error al búfer de salida y luego se calcula su longitud con `strlen`. La alerta indica que si el mensaje de error de la biblioteca subyacente no estuviera terminado en NUL, `strlen` leería fuera de los límites del búfer. Aunque es improbable que estas bibliotecas devuelvan cadenas mal formadas, la confianza implícita en un componente externo representa un riesgo teórico.
- **Explotabilidad: Baja**. Depende de un fallo en una biblioteca externa de confianza (como zlib) que resulte en un mensaje de error no terminado en NUL.

Línea 700 - CWE-126 - `makeerror` - FP

- **Código:**

```
*buf = RCAST(unsigned char *, msg);
*len = strlen(msg);
```

- **Análisis:** `msg` es el resultado de `vasprintf`, que garantiza la terminación NUL de la cadena que genera. Por lo tanto, la llamada a `strlen(msg)` es segura.
- **Explotabilidad:** Nula.

Línea 778 - CWE-126 - `filter_error` - FP

- **Código:**

```
++p;
while (isspace(CAST(unsigned char, *p)))
    p++;
n = strlen(p);
```

- **Análisis:** `p` es un puntero que se mueve dentro del búfer `ubuf`, que previamente ha sido terminado en NUL con `ubuf[n] = '\0'`. La lógica de la función no elimina esta terminación, por lo que la llamada a `strlen` sobre `p` (que apunta dentro de `ubuf`) es segura.
- **Explotabilidad:** Nula.

Archivo: file.c

Línea 608, 678, 681 - CWE-134 - `usage`, `help` - FP

- **Código:**

```
(void)fprintf(stderr, USAGE, pn, pn, pn); // 608
fprintf(stdout, "  -%c, --" longname, shortname), // 678
```

- **Análisis:** Flawfinder alerta sobre un posible ataque de cadena de formato. Sin embargo, en todos estos casos, la cadena de formato es un literal constante (`USAGE` es una macro a una cadena constante) definido por el programador. No hay forma de que un atacante pueda controlar el formato de la cadena.
- **Explotabilidad:** Nula.

Línea 712, 728, 742, 759 - CWE-134 - `file_err`, `file_errx`, `file_warn`, `file_warnx` - FP

- **Código:**

```
vfprintf(stderr, fmt, ap);
```

- **Análisis:** Estas funciones son envoltorios (`wrappers`) que usan `vfprintf`. La herramienta las marca porque el formato `fmt` es una variable. Sin embargo, estas funciones de utilidad son llamadas desde otras partes del programa siempre con cadenas de formato constantes y definidas en el código. El programa no usa datos de entrada (ej. de un archivo) como cadena de formato para estas funciones.
- **Explotabilidad:** Nula.

Línea 64, 216 - CWE-120 - `main` - FP

- **Código:**

```
int getopt_long(int, char * const *, const char *,... // 64
while ((c = getopt_long(argc, argv, OPTSTRING, long_options,... // 216
```

- **Análisis:** La alerta advierte sobre posibles desbordamientos de búfer en implementaciones antiguas de `getopt_long`. En cualquier sistema moderno que use una biblioteca C estándar (como glibc), esta función es segura y no presenta esta vulnerabilidad. La advertencia es más bien de carácter histórico.
- **Explotabilidad:** Nula.

Línea 213 - CWE-807 - `main` - VP

- **Código:**

```
posixly = getenv("POSIXLY_CORRECT") != NULL;
```

- **Análisis:** El programa modifica su comportamiento basándose en el valor de la variable de entorno `POSIXLY_CORRECT`. Un atacante que pueda establecer variables de entorno para el proceso puede alterar el flujo del programa, por ejemplo, para cambiar cómo se manejan los enlaces simbólicos.
- **Explotabilidad: Baja**. El impacto es limitado a cambiar un modo de operación del programa, no permite ejecución de código o escalada de privilegios directa.

Línea 462 - CWE-190 - `setparam` - VP

- **Código:**

```
pm[i].value = atoi(s + 1);
```

- **Análisis:** La función `atoi` convierte una cadena (leída del argumento de línea de comandos `-P`) a un entero. `atoi` no maneja desbordamientos; un número muy grande en la entrada puede causar un "wrap-around" y convertirse en un valor negativo o inesperado. Este valor se usa para configurar parámetros como `bytes_max`. Un valor malicioso podría llevar a asignaciones de memoria incorrectas o a un comportamiento anómalo.
- **Explotabilidad: Media**. Un atacante podría, a través de un valor cuidadosamente diseñado, provocar una denegación de servicio (DoS) por una asignación de memoria masiva o un comportamiento erróneo del análisis.

Línea 508 - CWE-362 - `unwrap` - VP

- **Código:**

```
if ((f = fopen(fn, "r")) == NULL) {
```

- **Análisis:** La variable `fn` contiene un nombre de archivo leído desde la línea de comandos o desde un archivo de lista (`-f`). El programa abre este archivo sin validar si es un enlace simbólico. Un atacante podría usar un enlace simbólico para hacer que `file` lea un archivo al que normalmente no debería tener acceso.
- **Explotabilidad: Baja**. Requiere que el atacante pueda crear un enlace simbólico en una ubicación donde `file` lo procesará y que el programa se ejecute con privilegios elevados para que el acceso a otros archivos sea significativo.

Línea 575, 600 - CWE-126 - `file_mbswidth` - FP

- **Código:**

```
old_n = n = strlen(s); // 575
return strlen(s); // 600
```

- **Análisis:** `strlen` se usa sobre la cadena `s` , que es un nombre de archivo. Los nombres de archivo pasados por el sistema operativo están garantizados a ser terminados en NUL. No hay riesgo de lectura fuera de límites en este contexto.
- **Explotabilidad:** Nula.

Archivo: magic.c

Línea 102, 203, 273, 276 - CWE-362/367 - `_w32_append_path` , `get_default_magic` , `unreadable_info` - VP

- **Código:**

```
if (access(tmppath, R_OK) == -1) // 102
if (access(hmagicpath, R_OK) == -1) // 203
if (access(file, W_OK) == 0) // 273
if (access(file, X_OK) == 0) // 276
```

- **Análisis:** Todas estas instancias representan una condición de carrera de tipo TOCTOU (Time-of-Check to Time-of-Use). El programa verifica la existencia o los permisos de un archivo con `access()` y luego realiza una operación basándose en ese resultado. Un atacante podría modificar el archivo (ej. reemplazarlo con un enlace simbólico) entre la llamada a `access()` y su uso posterior.

- **Explotabilidad: Baja**. La explotación es difícil de temporizar y el impacto depende del contexto. Por ejemplo, en `get_default_magic`, podría llevar a cargar un archivo de "magia" incorrecto, pero no a una ejecución de código.

Línea 188, 225, 229, 234, 255 - CWE-807 - `get_default_magic`, `magic_getpath` - VP

- **Código:**

```
if ((home = getenv("HOME")) == NULL) // 188
if ((home = getenv("LOCALAPPDATA")) != NULL) // 225
if ((home = getenv("USERPROFILE")) != NULL) // 229
if ((home = getenv("COMMONPROGRAMFILES")) != NULL) // 234
magicfile = getenv("MAGIC"); // 255
```

- **Análisis:** El programa construye la ruta a los archivos de bases de datos mágicas (`magic.mgc`) usando variables de entorno. Un atacante que controle el entorno del proceso puede establecer estas variables (`HOME`, `MAGIC`, etc.) para que apunten a una ruta bajo su control, forzando al programa a cargar una base de datos maliciosa. Esto puede causar DoS, identificación incorrecta de archivos o agotar recursos.
- **Explotabilidad: Media**. Es un vector de ataque viable en escenarios donde un atacante puede controlar el entorno de ejecución, como en servicios web que llaman a `file` como un proceso hijo.

Línea 443 - CWE-362 - `file_or_fd` - VP

- **Código:**

```
if ((fd = open(inname, flags)) < 0) {
```

- **Análisis:** Similar a la alerta en `file.c:508`. La variable `inname` es un nombre de archivo proporcionado por el usuario. El programa lo abre sin manejar de forma segura los enlaces simbólicos, lo que podría permitir a un atacante engañar al programa para que analice un archivo sensible.
- **Explotabilidad: Baja**. El impacto principal es la divulgación de información (el tipo de un archivo sensible), y requiere que `file` se ejecute con privilegios suficientes.

Línea 146 - CWE-126 - `_w32_get_magic_relative_to` - FP

- **Código:**

```
sp = strlen(dllpath);
```

- **Análisis:** La variable `dllpath` es un búfer que se llena mediante la llamada a la función de la API de Windows `GetModuleFileNameA`, la cual garantiza que la cadena resultante esté terminada en NUL. Por lo tanto, `strlen` es seguro.
- **Explotabilidad:** Nula.

Línea 507 - CWE-120 - `file_or_fd` - FP

- **Código:**

```
if ((nbytes = read(fd, RCAST(void *, buf), howmany)) == -1) {
```

- **Análisis:** La llamada a `read` intenta leer `howmany` bytes en el búfer `buf`. El tamaño de `buf` es `ms->bytes_max + SLOP` y el valor de `howmany` está limitado a `ms->bytes_max`. La lectura está dentro de los límites del búfer.
- **Explotabilidad:** Nula.

Archivo: print.c

Línea 242 - CWE-134 - `file_magwarn` - FP

- **Código:**

```
(void) vfprintf(stderr, f, va);
```

- **Análisis:** Similar a las alertas en `file.c`, esta función es un `wrapper` para `vfprintf`. La cadena de formato `f` proviene de las llamadas a `file_magwarn`. En todo el código, estas llamadas se realizan con cadenas de formato literales y constantes, no con datos controlados por el atacante.
- **Explotabilidad:** Nula.

Archivo: funcs.c

Línea 575 - CWE-120 - `file_getbuffer` - FP

- **Código:**

```
(void)memcpy(np, op, bytesconsumed);
```

- **Análisis:** La herramienta alerta de un posible desbordamiento en `memcpy`. Sin embargo, el búfer de destino `np` (`ms->o.pbuf`) se asigna con un tamaño 4 veces mayor que el búfer de origen `op` (`ms->o.buf`). La lógica de la función convierte caracteres no imprimibles en secuencias de escape de 4 bytes (`\123`). La asignación es suficiente para el peor caso, por lo que no hay desbordamiento.
- **Explotabilidad:** Nula.

Línea 267, 540 - CWE-126 - `trim_separator`, `file_getbuffer` - FP

- **Código:**

```
l = strlen(ms->o.buf); // 267
len = strlen(ms->o.buf); // 540
```

- **Análisis:** La variable `ms->o.buf` es el búfer de salida interno de la biblioteca. Todas las funciones que escriben en él (`file_printf`, `file_vprintf`) se aseguran de que esté correctamente terminado en NUL. Las llamadas a `strlen` sobre este búfer son seguras.
- **Explotabilidad:** Nula.

4.2 Analisis a cppcheck

Aquí tienes el análisis sistemático de los 112 problemas reportados por Cppcheck.

Problemas criticos

compress.c:400 - `error` - `uninitvar` - VP

- **Código afectado:** C

```
FD_ZERO(&check);
FD_SET(fd, &check);

/* ... */
selrv = select(fd + 1, &check, NULL, NULL, &tout);
```

- **Análisis:** Cppcheck reporta que la variable `check` (de tipo `fd_set`) se usa sin ser inicializada. Sin embargo, la línea `FD_ZERO(&check);` inmediatamente anterior a su uso sí la inicializa. Este es un **falso positivo de la herramienta**. Cppcheck falló al no reconocer `FD_ZERO` como una macro de inicialización.
- **Clasificación:** Falso Positivo (FP). La variable está correctamente inicializada.

---

print.c:55 - `warning` - `invalidPrintfArgType_uint` - VP

- **Código afectado:** C

```
(void) fprintf(stderr, "%u: %.*s %u", m->lineno,
    (m->cont_level & 7) + 1, ">>>>>>>>", m->offset);
```

- **Análisis:** El especificador de formato `%u` espera un `unsigned int`, pero la variable `m->offset` es de tipo `uint32_t` que, dependiendo de la arquitectura, puede ser un `unsigned long`. Esto puede causar un comportamiento indefinido.
- **Clasificación:** Verdadero Positivo (VP). Es un problema de portabilidad y corrección que, aunque de bajo riesgo, podría imprimir valores incorrectos en ciertas arquitecturas. No es directamente explotable.

---

print.c:65 - `warning` - `invalidPrintfArgType_uint` - VP

- **Código afectado:** C

```
(void) fprintf(stderr, "%c%u),",
    (CAST(size_t, m->in_op & FILE_OPS_MASK) <
    __arraycount(optyp)) ?
    optyp[m->in_op & FILE_OPS_MASK] : '?', m->in_offset);
```

- **Análisis:** Al igual que en el caso anterior, se usa `%u` para imprimir `m->in_offset`, que es de tipo `uint32_t`.
- **Clasificación:** Verdadero Positivo (VP). Misma justificación que en la línea 55.

---

print.c:137 - `warning` - `invalidPrintfArgType_sint` - VP

- **Código afectado:** C

```
(void) fprintf(stderr, "%d", m->value.l);
```

- **Análisis:** El especificador `%d` espera un `int` con signo, pero `m->value.l` se usa para múltiples tipos, incluyendo `unsigned int` en algunos casos. Esto puede llevar a una interpretación incorrecta del valor.
- **Clasificación:** Verdadero Positivo (VP). Es un error de tipo que puede resultar en la visualización de datos incorrectos.

---

funcs.c:197 - `error` - `syntaxError` - FP

- **Código afectado:** C

```
__attribute__((__format__(__printf__, 3, 0)))
```

- **Análisis:** Cppcheck reporta un error de sintaxis en un atributo específico de GCC/Clang. Este atributo se usa para que el compilador verifique que las llamadas a esta función usen cadenas de formato y argumentos de manera segura, similar a `printf`. El código es válido para estos compiladores.
- **Clasificación:** Falso Positivo (FP). Es una limitación de Cppcheck al no interpretar correctamente una extensión del compilador.

Problemas informáticos con implicaciones de seguridad

Tras revisar los 107 problemas restantes (`style`, `information`, `portability`), **no se encontró ninguno que oculte una vulnerabilidad de seguridad crítica**. La mayoría son:

- **`missingInclude`**: Informativo. Cppcheck no necesita las cabeceras del sistema, por lo que no afecta la calidad del análisis. 1
- **`unusedFunction`**: Problema de código muerto, no de seguridad. Indica que hay funciones que no se llaman, lo que sugiere que podrían eliminarse para reducir la superficie de código. 2
- **`variableScope`**: Sugerencia de estilo para mejorar la legibilidad y reducir el ámbito de las variables, sin impacto en la seguridad. 333333333333333333333333
- **`constParameterPointer`** / **`constVariablePointer`**: Sugerencias de estilo para usar `const` y mejorar la robustez del código. 44444444444444444444444444444444444

El único hallazgo de

portabilidad, `print.c:250`, indica un posible problema de conversión de tipos en `printf` (`%jd` con `uintmax_t`), lo cual es un bug de bajo impacto similar a los `warnings` ya analizados. 5

Resumen estadistico

- **Total analizado:** 112
- **Verdaderos Positivos:** 3
- **Falsos Positivos:** 2
- **Informativos relevantes para seguridad:** 0

4.3 Analisis General Falsos Negativos

Falso negativo #1:

- **Ubicación:** `apprentice.c:1352` (función `coalesce_entries`)
- **CWE:** `CWE-190` (Integer Overflow to Buffer Overflow)
- **Código problemático:** C

```
mentrycount = 0;
for (i = 0; i < nme; i++) {
    mentrycount += me[i].cont_count;
}
// ...
slen = sizeof(**ma) * mentrycount;
if ((*ma = CAST(struct magic *, malloc(slen))) == NULL) {
    // ...
}
// ...
mentrycount = 0;
for (i = 0; i < nme; i++) {
    (void)memcpy(*ma + mentrycount, me[i].mp,
        me[i].cont_count * sizeof(**ma));
    mentrycount += me[i].cont_count;
}
```

- Descripción: Existe una vulnerabilidad crítica de desbordamiento de entero que lleva a un desbordamiento de búfer en el heap. La variable mentrycount acumula la suma de cont_count de múltiples entradas mágicas. Un atacante puede crear un archivo de base de datos "magic" malicioso con varias entradas cuyos valores de cont_count sumen un número que exceda la capacidad de un uint32_t. Esto causará un "wrap-around" (desbordamiento), y mentrycount contendrá un valor final pequeño. Consecuentemente, malloc asignará un búfer muy pequeño (slen). Sin embargo, el segundo bucle memcpy utilizará los valores originales (grandes) de me[i].cont_count, escribiendo masivamente fuera de los límites del búfer asignado y corrompiendo el heap.
- **Por qué no detectado:** Las herramientas estáticas a menudo fallan en detectar desbordamientos de enteros que ocurren por acumulación en un bucle. No rastrean el valor de `mentrycount` a través de la iteración para predecir que la suma puede desbordarse antes de ser usada en `malloc`.
- **Explotabilidad: Alta**. Permite un desbordamiento de heap totalmente controlado a través de un archivo `magic.mgc` malicioso, lo que podría llevar a la ejecución de código arbitrario.
- **Método detección manual:** Auditar el código que procesa los archivos de base de datos mágica, buscando patrones donde se acumulen tamaños leídos del archivo y luego se usen para asignaciones de memoria. Específicamente, buscar el patrón `suma += valor_del_archivo` seguido de `malloc(suma * constante)`.

Falso negativo #2:

- **Ubicación:** `cdf.c:1194` (función `cdf_unpack_catalog`)

- **CWE:** `CWE-190` (Integer Overflow to Buffer Overflow)
- **Código problemático:** C

```
for (nr = 0;; nr++) {
    memcpy(&reclen, b, sizeof(reclen));
    // ... reclen se lee del archivo ...
    if (reclen == 0)
        break;
    b += reclen;
    if (b > eb)
        break;
}

*cat = CAST(cdf_catalog_t *,
    CDF_MALLOC(sizeof(cdf_catalog_t) + nr * sizeof(*ce)));
// ...
b = CAST(const char *, sst->sst_tab);
for (j = i = 0; i < nr; b += reclen) {
    // ... se lee reclen de nuevo ...
    memcpy(&cep->f, b + (l), sizeof(cep->f)); // Escritura fuera de
límites
}
```

- Descripción: Similar al caso anterior, esta función procesa un catálogo dentro de un archivo CDF (Composite Document File). Primero, calcula el número de registros (nr) iterando sobre el búfer. Un atacante puede crear un archivo con un gran número de registros de tamaño mínimo para que nr alcance un valor muy grande. La multiplicación nr *sizeof(*ce) en la llamada a CDF_MALLOC puede desbordarse, resultando en una asignación de memoria insuficiente. El segundo bucle, que itera nr veces, procederá a escribir metadatos de los registros fuera de los límites del búfer del catálogo asignado, corrompiendo el heap.
- **Por qué no detectado:** La lógica es compleja. El valor de `nr` se determina en un primer bucle y luego se usa para una asignación en una expresión aritmética. Las herramientas no conectaron el origen de los datos del archivo con el cálculo del tamaño de la asignación y el uso posterior en el segundo bucle.
- **Explotabilidad: Alta**. Un archivo CDF malicioso puede desencadenar un desbordamiento de heap controlable, potencialmente llevando a la ejecución de código.
- **Método detección manual:** Analizar los formatos de archivo complejos (como CDF). Identificar bucles que calculan el número de entradas (`nr`) basándose en metadatos del propio archivo. Luego, verificar si `nr` se usa en una multiplicación para asignar memoria y si un desbordamiento en esa multiplicación resulta en una escritura fuera de límites en bucles posteriores.

Falso negativo #3:

- **Ubicación:** `readelf.c:1208` (función `donote`)
- **CWE:** `CWE-680` (Integer Overflow to Unbounded Loop)
- **Código problemático:** C

```
namesz = xnh_namesz;
descsz = xnh_descsz;
// ...
doff = ELF_ALIGN(noff + namesz);
offset = ELF_ALIGN(doff + descsz);

if (doff + descsz > size) { // El cheque es insuficiente
    return (offset >= size) ? offset : size;
}
```

- Descripción: Al procesar las secciones de notas de un archivo ELF, las variables namesz y descsz se leen directamente de la cabecera de la nota. Un atacante puede establecer un valor muy grande para descsz. La suma doff + descsz puede desbordar y "envolver" a un valor pequeño, haciendo que el cheque de seguridad doff + descsz > size sea superado. Sin embargo, la variable offset se calcula con ELF_ALIGN(doff + descsz), que también usará el valor desbordado. Esto puede resultar en que offset sea menor que su valor anterior, haciendo que el bucle que procesa las notas (en funciones como doshn) se vuelva infinito o procese datos incorrectos, llevando a una denegación de servicio (DoS).
- **Por qué no detectado:** Detectar desbordamientos de enteros en operaciones de suma que luego afectan el flujo de control (como el incremento de un puntero en un bucle) es un desafío para el análisis estático. Las herramientas no evaluaron el posible "wrap-around" de la suma `doff + descsz`.
- **Explotabilidad: Media**. No permite la ejecución de código directamente, pero un archivo ELF malicioso puede hacer que el programa `file` entre en un bucle infinito, consumiendo el 100% de la CPU y causando una denegación de servicio.
- **Método detección manual:** Buscar código que parsea formatos binarios donde un tamaño (`descsz`) leído del archivo se suma a un offset (`doff`). Investigar si la suma puede desbordarse y si el resultado se usa para calcular un nuevo offset o para una validación de límites.

Falso negativo #4:

- **Ubicación:** `readelf.c:1389` (función `doshn`)
- **CWE:** `CWE-125` (Out-of-bounds Read)
- **Código problemático:** C

```
// name_off se calcula a partir del campo sh_offset de la cabecera de
la tabla de strings.
name_off = xsh_offset;
// ...
for (num_copy = num; num_copy; num_copy--, i++) {
    // ... se lee una cabecera de sección ...
    // xsh_name es el offset *dentro* de la tabla de strings
    offs = name_off + xsh_name;
    if ((namesize = pread(fd, name, sizeof(name) - 1, offs))
        == -1) {
        // ...
    }
    name[namesize] = '\0';
    // ... se usa 'name' ...
}
```

- Descripción: Esta función lee las cabeceras de las secciones de un archivo ELF. Para obtener el nombre de cada sección, combina un offset base (name_off, que es el inicio de la tabla de nombres de sección) con un offset relativo (xsh_name) leído de la cabecera de la sección actual. Ambos valores son controlables por un atacante. Un atacante puede crear un archivo ELF donde la combinación name_off + xsh_name apunte a cualquier lugar del archivo. Esto fuerza a pread a leer datos de una ubicación arbitraria. Aunque se lee en un búfer de tamaño fijo (name), esto constituye una vulnerabilidad de fuga de información, ya que permite leer y mostrar fragmentos de cualquier parte del archivo que se está analizando.
- **Por qué no detectado:** El cálculo del offset de lectura (`offs`) depende de dos valores (`name_off` y `xsh_name`) que se obtienen en diferentes momentos del parsing. Las herramientas de análisis estático a menudo no pueden realizar el análisis de flujo de datos necesario para determinar que la combinación de estas dos variables controladas por el atacante puede resultar en un puntero arbitrario.
- **Explotabilidad: Media**. No permite ejecución de código, pero sí la lectura de partes arbitrarias de un archivo. Si `file` se ejecuta sobre un archivo sensible con una estructura ELF maliciosa, podría revelar información confidencial.
- **Método detección manual:** Revisar todo el código que realiza operaciones de lectura (`read` , `pread` , `memcpy`) donde el offset o puntero de origen se calcula a partir de más de una variable. Rastrear el origen de cada una de esas variables para ver si provienen de datos de entrada no confiables.

Falso negativo #5:

- **Ubicación:** `funcs.c:145` (función `file_vprintf`)

- **CWE:** `CWE-190` (Integer Overflow)
- **Código problemático:** C

```
len = vasprintf(&buf, fmt, ap);
if (len < 0 || (size_t)len > 1024 || len + ms->o.blen > 1024 * 1024) {
    size_t blen = ms->o.blen;
    free(buf);
    file_clearbuf(ms);
    file_error(ms, 0, "Output buffer space exceeded %d+%zu", len,
blen);
    return -1;
}
```

- Descripción: Esta función construye la cadena de salida de file. Hay una comprobación para limitar el tamaño total a 1MB (1024 * 1024). Sin embargo, la comprobación len + ms->o.blen es vulnerable a un desbordamiento de entero. Si ms->o.blen (la longitud actual del búfer) ya es grande, y una nueva llamada a file_printf genera una cadena de longitud len también grande, la suma len + ms->o.blen puede desbordar (hacer "wrap-around") y resultar en un número pequeño. Este número pequeño pasaría la validación < 1MB. La siguiente llamada a asprintf intentaría asignar una cantidad de memoria gigantesca (cercana al máximo de size_t), lo que resultaría en un fallo de asignación de memoria y una denegación de servicio (DoS).
- **Por qué no detectado:** Los analizadores estáticos a menudo no modelan el comportamiento de desbordamiento de la aritmética de enteros. Ven una comprobación de seguridad (`+ >`) y asumen que es suficiente, sin considerar que la propia operación de suma puede desbordarse y anular la lógica de la comprobación.
- **Explotabilidad: Baja**. Requiere que una regla de `magic` maliciosa construya una cadena de salida extremadamente larga de forma incremental para provocar el desbordamiento. El resultado es una DoS, no ejecución de código.
- **Método detección manual:** Auditar todas las comprobaciones de seguridad que involucren sumas de tamaños (`offset + len` , `len1 + len2`). Evaluar si un atacante puede controlar ambos operandos para provocar un desbordamiento que invalide la comprobación. Es un patrón común en código C que maneja tamaños y búferes.

Tabla de vulnerabilidades generada con LLM Gemini

Tabla Consolidada de Vulnerabilidades en 'file' v5.41

#	Línea	Debilidad	LLM	Explicación	Flawfinder	Cpp
1	`file.c:213`	`CWE-807`	**VP**	El comportamiento del programa puede ser alterado por un atacante que controle la variable de entorno `POSIXLY_CORRECT`.	✔	
2	`file.c:462`	`CWE-190`	**VP**	`atoi` no verifica desbordamientos, permitiendo un integer overflow que puede llevar a asignaciones de memoria erróneas.	✔	
3	`file.c:508`	`CWE-362`	**VP**	Condición de carrera (TOCTOU): `fopen` sobre una ruta controlada por el usuario es vulnerable a ataques de enlaces simbólicos.	✔	
4	`file.c:608`	`CWE-134`	**FP**	Falso positivo. La cadena de formato usada en `fprintf` es un literal constante, no es controlable por el atacante.	✔	
5	`magic.c:255`	`CWE-807`	**VP**	`getenv("MAGIC")` permite a un atacante especificar una base de datos maliciosa, causando DoS o análisis incorrectos.	✔	

#	Línea	Debilidad	LLM	Explicación	Flawfinder	Cpp
6	`magic.c:102`	`CWE-367`	**VP**	Condición de carrera (TOCTOU): uso inseguro de `access` antes de una operación de archivo, vulnerable a ataques de enlaces simbólicos.	✔	
7	`magic.c:443`	`CWE-362`	**VP**	`open` sobre un nombre de archivo de entrada del usuario es vulnerable a condiciones de carrera (TOCTOU).	✔	
8	`compress.c:455`	`CWE-377`	**VP**	Condición de carrera clásica usando `mktemp` (en código legacy), permitiendo a un atacante redirigir la creación de archivos.	✔	
9	`compress.c:595`	`CWE-126`	**VP**	`strlen` puede leer fuera de límites si una biblioteca de descompresión externa devuelve un mensaje de error sin terminación NUL.	✔	
10	`compress.c:909`	`CWE-78`	**FP**	Falso positivo. `execvp` usa comandos predefinidos; un atacante solo elige cuál se ejecuta, no inyecta comandos arbitrarios.	✔	

#	Línea	Debilidad	LLM	Explicación	Flawfinder	Cpp
11	`print.c:55`	`CWE-134`	**VP**	Discrepancia de tipos en `fprintf` (`%u` con `int`), causando comportamiento indefinido y visualización incorrecta de datos.		✔
12	`funcs.c:145`	`CWE-190`	**FN**	Desbordamiento de entero en el cheque `len + ms->o.blen`, permitiendo evadir el límite y causar DoS por `malloc`.		
13	`apprentice.c:1352`	`CWE-190`	**FN**	Desbordamiento de entero al sumar contadores de un archivo 'magic' malicioso, llevando a un desbordamiento de heap masivo.		
14	`cdf.c:1194`	`CWE-190`	**FN**	Desbordamiento de entero al contar registros en un archivo CDF malicioso, resultando en un desbordamiento de heap.		
15	`readelf.c:1208`	`CWE-680`	**FN**	Desbordamiento de entero al calcular offsets en notas ELF, lo que puede causar un bucle infinito y denegación de servicio (DoS).		

#	Línea	Debilidad	LLM	Explicación	Flawfinder	Cpp
16	`readelf.c:1389`	`CWE-125`	**FN**	Lectura fuera de límites (Out-of-bounds Read) al combinar offsets controlados por el atacante, permitiendo fuga de información.		

5. Reflexion

A mi parecer las herramientas de análisis de codigo son herramientas extremadamente utiles, ya que son capaces de identificar patrones de manera lo suficientemente rapida en cantidades de codigo de gran tamaño. En este caso solo se analizaron 3500 lineas de codigo, pero, cuando se intentó analizar la totalidad del software file la herramienta flawfinder analizó entre 16000 y 21000 lineas de código en un tiempo considerablemente reducido.

A pesar de que estas herramientas son veloces, versatiles y, sobretodo, útiles, no son perfectas y es posible encontrar alguno que otro fallo en sus análisis. Las herramientas como las utilizadas operan sobre patrones predefinidos y fáciles de detectar, lo que genera la eficiencia mencionada anteriormente, pero esto también las hace caer en errores a los que son susceptibles los algoritmos como puede ser la falta de comprensión contextual. Esta falta puede generar una gran cantidad de falsos positivos o debilidades de una importancia inflada debido a que se notan patrones que, bajo cualquier otro contexto, suelen significar una debilidad potencialmente peligrosa, pero no en el contexto especifico en el que se está analizando. Gracias a estos pequeños fallos estas herramientas necesitan mucho trabajo de post-analisis para descartar todos esas falsas alarmas.

Las herramientas pueden ser muy útiles en la identificación de ciertas debilidades especificas y mediocres en la detección de otras, ya que existen debilidades que, con mas frecuencia que las demás tienen un patrón evidente mientras que están las que dependen meramente de suposiciones que una herramienta de análisis de código estático no puede efectuar.

Dado lo anteriormente expuesto se puede llegar a la conclusión de que muy buenas para una primera inspección, pero no son un método definitivo y es siempre necesario complementarlo de un post-analisis, ya sea con herramientas LLM, otro tipo de herramientas o de forma manual si fuera posible. Esto para conseguir un análisis mas acertado y evitando, sobre todo, destinar recursos a la corrección de falsos positivos y, especialmente, no dejar por fuera falsos negativos que no hayan sido captados en los patrones de los analizadores automáticos.

6. Recomendaciones

a. Adecuarse a las herramientas antes de empezar a trabaja en el analisis

Durante el desarrollo del proyecto se dieron varias complicaciones relacionadas a las herramientas, entre ellas el problema de incompatibilidad entre 2 de ellas y el sistema operativo en el que se estaba trabajando. La previa búsqueda de información sobre la compatibilidad de las herramientas podría haber haber evitado el inicio del proyecto en un sistema operativo incompatible y la mala experiencia.

Ademas se tuvieron que hacer mas de un análisis con las herramientas debido a la ignorancia de su forma de funcionamiento, ya que, es extremadamente personalizable y algo enrevesada de entender al utilizarse únicamente por consola.

b. Utilizar estrategias de selección inteligente de código

En el caso especifico de file hay mucho código no central en el funcionamiento del software. Debido a que en este proyecto se exigía una cantidad de lineas de código mínimas no fue de interés hacer un análisis exhaustivo a todo ese código secundario, por lo que se tuvo que hacer un descarte por medio criterios de importancia para seleccionar a lo que se le llamó nucleo funcional y analizar solamente eso.

c. Definir que se va a documentar con antelación

Al generar los archivos de reporte me encontraba con bloques bastante grandes de texto difíciles de analizar debido a la configuración de las herramientas, por lo que fue necesario extraer los datos especificos de estos archivos reporte generados por medio de otras herramientas para que al LLM no le consumiera demasiada memoria el procesarlos para el análisis del código

d. Documentar adecuadamente el proceso

Una de las cosas que facilitó mucho el post análisis con la herramienta LLM fue la previa documentación de los resultados dados por la herramienta. Por motivos de contexto, fácil acceso a la información para la herramienta LLM y una menor cantidad de información a procesar para el LLM, se puede recomendar el tener una documentación adecuada de los resultados de los análisis.

e. Evitar atascar al LLM con toda la información

Los LLM suelen omitir información cuando su memoria se llena de datos que analizar y empiezan a generalizar. Esto puede generar que de un resultado mas ineficiente, por lo que podria ser conveniente analizar el codigo en pedazos pequeños en lugar de unos mas grandes a pesar de que sea un proceso mucho mas tardado.